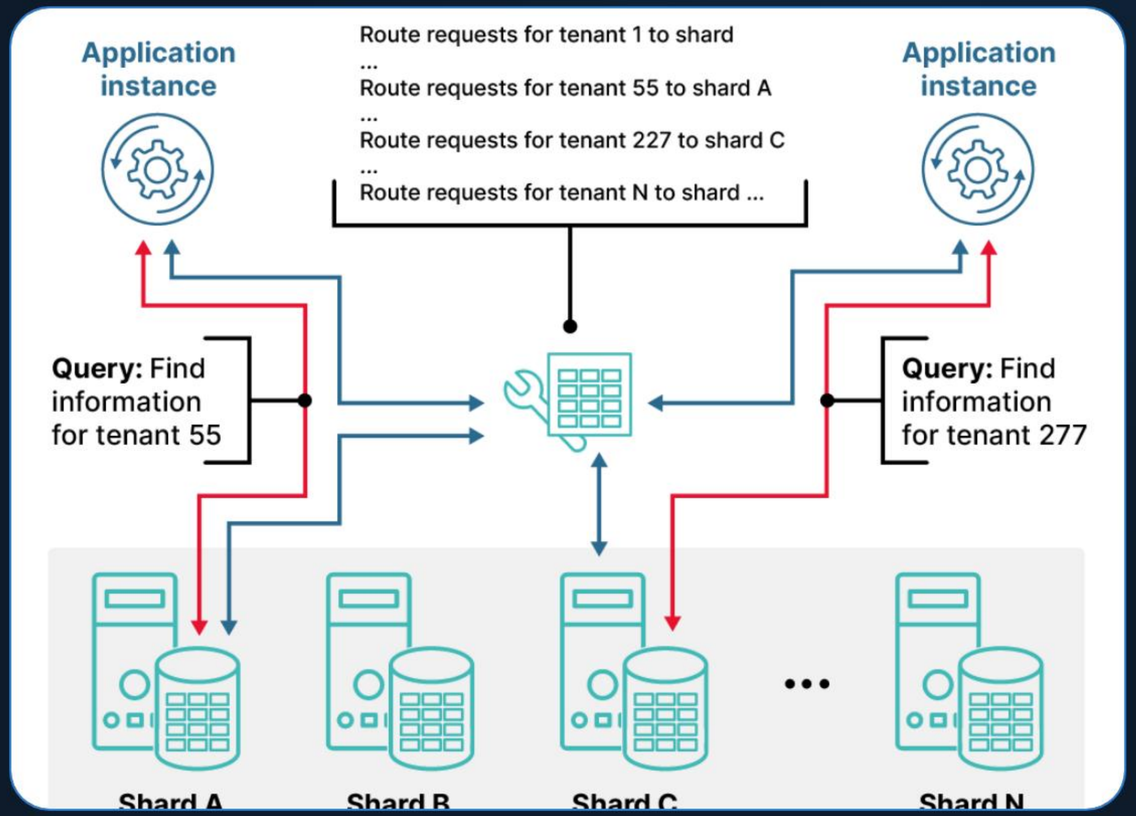
SOFTWARE ARCHITECTURE SERIES

The Publish-Subscribe Pattern

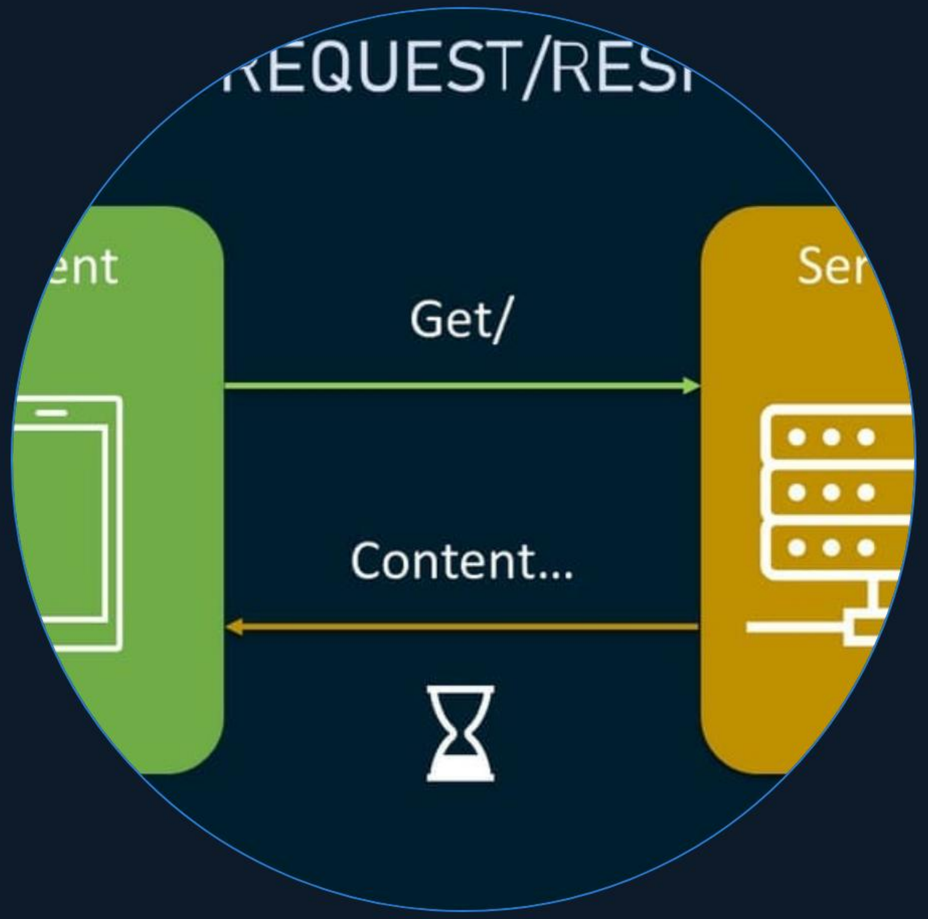
Asynchronous Messaging at Scale

The Synchronous Challenge

- **Tight Coupling:** Senders must know specific receiver endpoints.
- **Blocking Calls:** Systems wait for responses, creating latency.
- **Cascading Failures:** One downed service halts the entire request chain.
- **Inelastic Scalability:** Hard to handle sudden traffic spikes.



What is Pub-Sub?



Indirect Communication

The Publish-Subscribe pattern is a messaging paradigm where senders (publishers) do not program the messages to be sent directly to specific receivers (subscribers).

Instead, messages are categorized into **classes or topics** without knowledge of which subscribers, if any, there may be.

How It Works

1. Publish

Publisher sends a message to a specific topic on the broker.

2. Route

Broker receives the message and identifies active subscriptions.

3. Filter

Messages are filtered by topic or content for each subscriber.

4. Deliver

Broker pushes messages to subscribers asynchronously.

Dimensions of Decoupling

Space Decoupling

Publishers and subscribers do not need to know each other's network location or identity.

Time Decoupling

Components do not need to be online at the same time to exchange information successfully.

Sync Decoupling

Publishers are not blocked while subscribers process the messages they've received.

Message Anatomy

The structure of a Pub-Sub packet

```
// Header: Metadata for Routing
{ "topic": "weather-updates", "sender": "service-a", "ts": 171717 }

// Body: The Data Payload
{ "temp": 72, "unit": "F", "alert": false }
```

Event Delivery Models

Event Notification

The message contains only a minimal notification (e.g., an ID). The subscriber must query a source to get full details.

- Low bandwidth usage
- Requires extra API calls

Carried State Transfer

The message includes all the data required for the subscriber to perform its task without further requests.

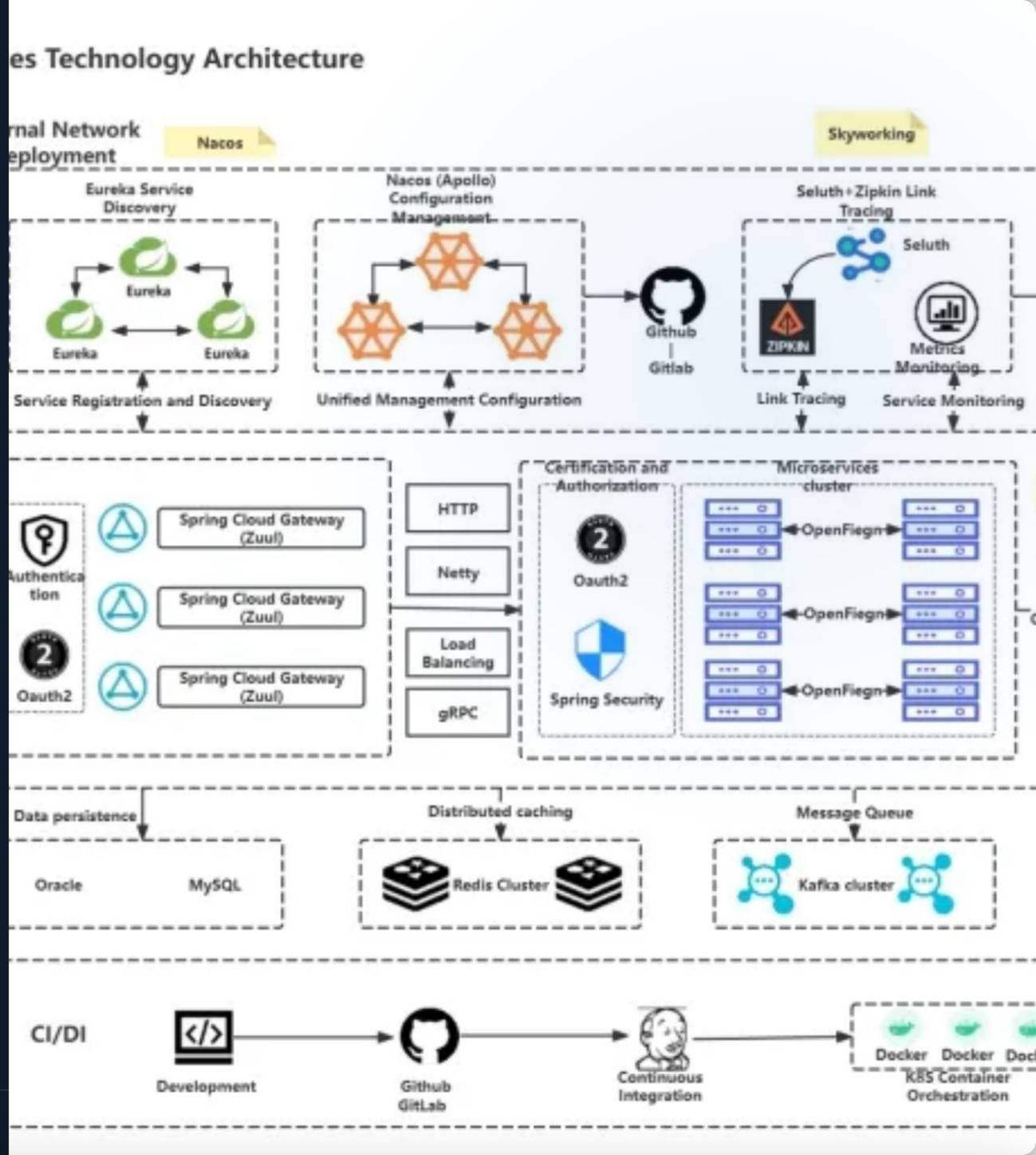
- High autonomy for systems
- Increases message size

Core Advantages

- **Fault Isolation:** A single failure doesn't crash the ecosystem.
- **Elastic Scaling:** Add consumers dynamically as load increases.
- **Responsiveness:** Senders finish tasks immediately after publishing.
- **Heterogeneous Tech:** Connect Python, Go, and JS services easily.

Critical Challenges

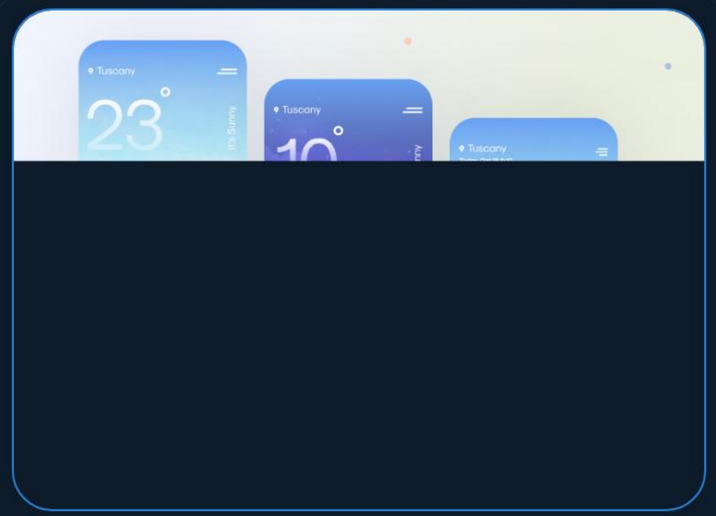
- **Debug Complexity:** Tracing events across services is non-trivial.
- **Ordering:** Message sequence is not guaranteed by default.
- **At-least-once:** Handling duplicate messages (idempotency).



Pub-Sub vs Point-to-Point

Feature	Publish-Subscribe	Point-to-Point (Queue)
Cardinality	One-to-Many (Broadcast)	One-to-One (Unicast)
Coupling	Completely Decoupled	Loosely Coupled
Typical Use	Updates & Notifications	Work Queues & Task Processing

Real-World Applications

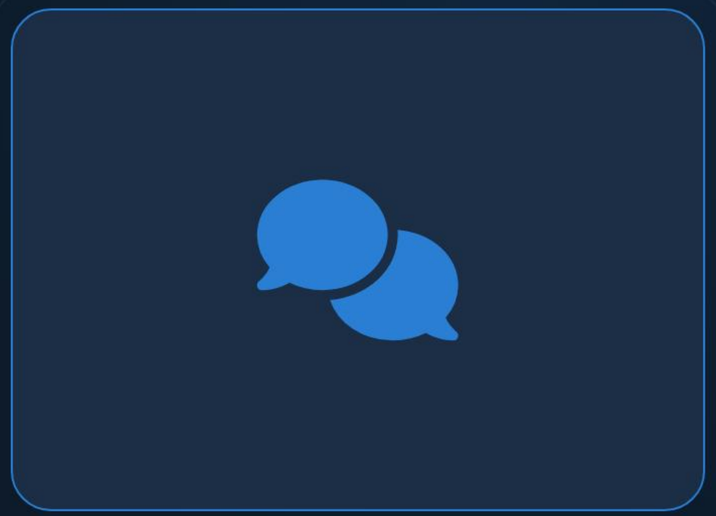


Weather Updates

Broadcasting data to millions of IoT devices.

Financial Services

Real-time stock tickers and trade notifications.



Live Chat

Synchronizing state across multiple web users.

Questions?

"Architecture is about making the important decisions now so that you don't have to worry about the details later."

Kafka

RabbitMQ

AWS SNS/SQS

Image Sources

https://www.solarwinds.com/wp-content/uploads/2022/01/What-is-a-distributed-system-distributed-system-sharded-pattern-example.png

Source: www.solarwinds.com

https://www.altexsoft.com/static/content-image/2024/7/374b1404-54e6-4ba1-8aae-0482237eeb05.jpg

Source: www.altexsoft.com

https://cdn.processon.io/admin/knowledge/article_content_img/67b6a344bcb4581cedd12229.png

Source: www.processon.io

https://cdn.dribbble.com/userupload/2581670/file/original-4556a028d4c549717c922d095f3e1401.png

Source: dribbble.com

https://assets.qlik.com/image/upload/w_2378/q_auto/qlik/glossary/dashboard-examples/seo-hero-financial-dashboards_ttulhs.png

Source: www.qlik.com
